



GUIA TECNICO COMPLETO

# Apache Kafka

Arquitetura, conceitos, garantias de entrega  
e pratica com Java

---

Topicos • Particoes • Producers • Consumers • Consumer Groups

Offsets • Replicacao • Retencao e Compactacao de Log

Comparativo com RabbitMQ e Amazon SQS/SNS • Exemplos praticos em Java

# Sumário

---

1. Introdução: por que Kafka é diferente
2. Arquitetura Fundamental
3. Producers
4. Consumers e Consumer Groups
5. Garantias de Entrega e Semânticas de Processamento
6. Retenção e Compactação de Log
7. Instalação e Ambiente de Desenvolvimento
8. Trabalhando com Kafka em Java
9. O Ecossistema Kafka (visão geral)
10. Boas Práticas e Erros Comuns
11. Apache Kafka vs. RabbitMQ vs. Amazon SQS/SNS
12. Próximos Passos

# 1. Introdução: por que Kafka é diferente

## 1.1. O ponto de partida: você já conhece um broker tradicional

Você já estudou RabbitMQ e SQS/SNS — dois exemplos de **brokers tradicionais**. Neles, a mensagem é entregue, o consumer confirma (ack), e a mensagem é **removida** da fila. O broker é essencialmente uma fila temporária: mensagens entram, são processadas, e desaparecem.

O Apache Kafka parte de uma premissa completamente diferente. Ele não é uma fila — é um **log distribuído, particionado e replicado**. Uma mensagem publicada no Kafka não é removida quando é lida; ela permanece armazenada até sua política de retenção expirar (por tempo ou por tamanho), independentemente de quantos consumers já a leram. Isso muda tudo: onde o RabbitMQ modela "trabalho a ser feito uma vez", o Kafka modela "um histórico de eventos que pode ser lido (e relido) por quantos interessados quiserem, cada um na sua própria velocidade".

### A analogia mais útil

Pense em uma fila do RabbitMQ como uma **caixa de entrada de e-mail**: cada mensagem é lida e arquivada/apagada. Pense em um tópico do Kafka como um **livro de registro (log) contínuo**, tipo a fita de um caixa eletrônico: os eventos ficam escritos em ordem, permanentemente (até expirar por retenção), e cada leitor mantém o seu próprio marcador de "até onde eu já li" — sem afetar o marcador de ninguém mais.

## 1.2. Mapeando o que você já sabe (RabbitMQ/SQS/SNS) para o Kafka

| Conceito RabbitMQ/AWS                      | Equivalente aproximado no Kafka                                            | Diferença importante                                                                                                               |
|--------------------------------------------|----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Fila (queue)                               | Partição de um tópico                                                      | A partição não remove a mensagem ao ler                                                                                            |
| Exchange                                   | Não existe um equivalente direto                                           | O roteamento é decidido pelo producer (qual tópico/partição), não por uma camada de exchange                                       |
| Ack manual (RabbitMQ)                      | Commit de offset                                                           | Você confirma "até onde li", não mensagem por mensagem                                                                             |
| Múltiplos workers competindo (Work Queue)  | Consumer group com múltiplas instâncias                                    | No Kafka, cada partição só pode ser lida por um consumer do grupo por vez — o paralelismo máximo é limitado ao número de partições |
| Fanout (múltiplos serviços recebendo tudo) | Múltiplos consumer groups diferentes lendo o mesmo tópico                  | Isso é nativo e barato no Kafka — cada grupo tem seu próprio offset independente                                                   |
| DLQ (Dead Letter Queue)                    | Não é nativo — implementado manualmente (tópico de erro + reprocessamento) | Kafka não tem dead-lettering automático como o RabbitMQ                                                                            |
| Mensagem removida após ack                 | Mensagem permanece até expirar por retenção                                | Isso permite <i>replay</i> : reprocessar o histórico do zero                                                                       |

## 1.3. Quando usar Kafka em vez de RabbitMQ

### Kafka brilha quando você precisa de:

- **Altíssimo throughput** — milhões de eventos por segundo, com garantias de ordenação por chave.
- **Replay de histórico** — reprocessar dados antigos (ex.: reconstruir um relatório, popular um novo serviço com o histórico completo de eventos).
- **Múltiplos consumidores independentes lendo o mesmo fluxo** — sem duplicar a mensagem N vezes fisicamente, como aconteceria com N filas fanout.
- **Event sourcing / streaming de dados** — o log é a fonte da verdade, não um efeito colateral.
- **Integração com processamento analítico** — Kafka Streams, ksqlDB, Spark Streaming, Flink.

#### **RabbitMQ ainda é melhor quando você precisa de:**

Roteamento sofisticado por conteúdo (topic/headers exchange), prioridade de mensagens, RPC síncrono, ou simplesmente um modelo mais simples de "processar uma vez e esquecer" sem se preocupar com particionamento e offsets.

## 2. Arquitetura Fundamental

### 2.1. Peças principais

| Termo                       | Definição                                                                                                                                |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Broker                      | Um servidor Kafka. Um cluster é composto por múltiplos brokers.                                                                          |
| Cluster                     | Conjunto de brokers trabalhando juntos, coordenados via KRaft (ou, em versões antigas, ZooKeeper).                                       |
| Topic (tópico)              | Uma categoria/nome de fluxo de eventos — o equivalente mais próximo de uma "fila", mas com semântica de log.                             |
| Partition (partição)        | Um tópico é dividido em N partições, cada uma um log ordenado e imutável, armazenado fisicamente em um broker.                           |
| Offset                      | Número sequencial que identifica a posição de uma mensagem dentro de uma partição.                                                       |
| Producer                    | Aplicação que publica (escreve) registros em um tópico.                                                                                  |
| Consumer                    | Aplicação que lê registros de um tópico, avançando seu próprio offset.                                                                   |
| Consumer Group              | Conjunto de consumers que dividem entre si a leitura das partições de um tópico.                                                         |
| Replica / Leader / Follower | Cada partição é replicada em múltiplos brokers; um é o líder (recebe leitura/escrita), os demais são seguidores (réplicas de segurança). |
| KRaft                       | Modo moderno do Kafka (substituindo o ZooKeeper) onde os próprios brokers, usando o protocolo Raft, gerenciam os metadados do cluster.   |

### 2.2. O fim do ZooKeeper: KRaft

Historicamente, o Kafka dependia do **Apache ZooKeeper** como serviço externo para coordenar metadados do cluster (quais brokers existem, qual é o líder de cada partição, configurações). Desde o Kafka 3.3 (KRaft marcado como pronto para produção) e obrigatório a partir da série 4.x, o Kafka usa o modo **KRaft** (Kafka Raft), onde um subconjunto dos próprios brokers atua como "controladores", usando o algoritmo de consenso Raft para gerenciar os metadados internamente — eliminando a dependência de um sistema externo. Para quem está aprendendo hoje, isso significa uma peça a menos para instalar e entender: você só precisa subir brokers Kafka, não um cluster ZooKeeper separado.

### 2.3. Tópicos e Partições em detalhe

Um tópico é dividido em uma ou mais partições no momento da criação (o número pode ser aumentado depois, mas não diminuído). Cada partição é um **log append-only**: novos registros são sempre escritos no final, nunca no meio, e nunca modificados depois de escritos.

```
Topico: pedidos.eventos  (3 particoes)

Particao 0: [msg0] [msg1] [msg2] [msg3] [msg4] ...
Particao 1: [msg0] [msg1] [msg2] ...
Particao 2: [msg0] [msg1] [msg2] [msg3] ...
                ^
                proximo offset livre
```

A ordem só é garantida **dentro de uma mesma partição**, nunca entre partições diferentes de um mesmo tópico. Isso é uma diferença crucial: se a ordem entre dois eventos importa (ex.: "criar pedido" antes de "cancelar pedido" do mesmo pedido), ambos precisam cair na **mesma partição** — o que é garantido publicando com a mesma **chave (key)**, já que o particionamento padrão é feito com base no hash da chave.

#### Por que particionar

Partições são a unidade de paralelismo do Kafka. Mais partições = mais consumers podem trabalhar em paralelo lendo o

mesmo tópico simultaneamente (até o limite de 1 consumer por partição, por grupo). É também a unidade de distribuição física: partições diferentes de um mesmo tópico podem viver em brokers diferentes, distribuindo carga de I/O pelo cluster.

## 2.4. Replicação: Leader e Followers

Cada partição tem um **fator de replicação** configurável (tipicamente 3 em produção). Isso significa que a mesma partição é copiada para múltiplos brokers: um é eleito **líder** (recebe todas as leituras e escritas daquela partição), e os demais são **seguidores (followers)**, que replicam passivamente os dados do líder. Se o broker líder cair, um dos seguidores é promovido automaticamente a novo líder — desde que esteja "em sincronia" (In-Sync Replica, ISR).

```
Particao 0 do topico "pedidos" (fator de replicacao = 3)

Broker 1: [LIDER]      - recebe todas as leituras/escritas
Broker 2: [FOLLOWER]  - replica passivamente do lider
Broker 3: [FOLLOWER]  - replica passivamente do lider

Se o Broker 1 cair -> Broker 2 ou 3 (o que estiver em ISR) vira o novo lider
```

## 3. Producers

### 3.1. Anatomia de um registro (record)

Um registro Kafka é composto por:

- **Key (chave):** opcional, mas crucial — determina para qual partição o registro vai (via hash), e portanto sua ordem relativa a outros registros com a mesma chave.
- **Value (valor):** o payload em si — bytes, sem formato imposto pelo Kafka (JSON, Avro, Protobuf, texto puro, etc).
- **Headers:** metadados customizados, similar aos headers do AMQP.
- **Timestamp:** momento da criação ou da ingestão, dependendo da configuração do tópico.
- **Partition e Offset:** atribuídos pelo Kafka no momento da escrita — o producer pode sugerir a partição, mas não o offset (que é sempre sequencial e automático).

### 3.2. Como a partição é escolhida

| Cenário                                    | Comportamento                                                                                                                                         |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Chave (key) informada                      | Kafka aplica hash da chave para escolher a partição — mesma chave sempre cai na mesma partição, garantindo ordem relativa entre eventos daquela chave |
| Sem chave (key = null)                     | Desde o Kafka 2.4+, o particionador padrão distribui em lotes ("sticky partitioning") para otimizar throughput, em vez de round-robin estrito         |
| Partição explícita informada pelo producer | O Kafka respeita e ignora a chave para fins de particionamento                                                                                        |

### 3.3. Publisher Acks: a garantia de durabilidade do producer

O parâmetro `acks` do producer controla o quão rigorosa é a confirmação de escrita:

| acks        | Significado                                                         | Trade-off                                                                           |
|-------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| 0           | Producer não espera confirmação nenhuma — "dispara e esquece"       | Maior throughput, risco real de perda de dados                                      |
| 1           | Producer espera confirmação apenas do broker líder                  | Equilíbrio — mas se o líder cair antes de replicar, a mensagem pode se perder       |
| all (ou -1) | Producer espera confirmação de todas as réplicas em sincronia (ISR) | Máxima durabilidade, menor throughput — recomendado em produção para dados críticos |

### 3.4. Idempotência do producer

Ao ativar `enable.idempotence=true` (padrão desde o Kafka 3.0 em diante quando `acks=all`), o Kafka atribui um ID único a cada producer e um número de sequência a cada mensagem, permitindo que o broker detecte e descarte automaticamente duplicatas causadas por reenvios de rede — sem esse recurso, um simples timeout de rede seguido de retry do producer poderia gravar a mesma mensagem duas vezes.

# 4. Consumers e Consumer Groups

## 4.1. O conceito central: consumer group

Um **consumer group** é um conjunto de consumers identificados pelo mesmo `group.id`, que dividem entre si a leitura das partições de um tópico. A regra fundamental:

### A regra que rege tudo neste capítulo

Dentro de um consumer group, **cada partição é lida por, no máximo, um consumer por vez**. Se você tem 6 partições e 3 consumers no grupo, cada consumer lê, em média, 2 partições. Se você tem 6 partições e 8 consumers, 2 consumers ficam ociosos — não existe "meia partição" para dividir.

Topico "pedidos" com 4 particoes, Consumer Group "processamento-pedidos"

Particao 0 ---> Consumer A  
Particao 1 ---> Consumer A  
Particao 2 ---> Consumer B  
Particao 3 ---> Consumer C

(3 consumers no grupo, 4 particoes: A ficou com 2, B e C com 1 cada)

## 4.2. Múltiplos grupos = múltiplos "fanouts" independentes

Se dois consumer groups diferentes ( `group.id` diferentes) lerem o mesmo tópico, cada grupo recebe **sua própria cópia completa** do fluxo de eventos, com seu próprio conjunto de offsets, totalmente independente um do outro. Isso é o que torna o Kafka naturalmente "fanout" sem esforço extra — diferente do RabbitMQ, onde você precisaria de uma exchange fanout com N filas fisicamente separadas.

## 4.3. Rebalanceamento (rebalance)

Quando um consumer entra ou sai de um grupo (nova instância subindo, uma caindo, ou um deploy), o Kafka redistribui as partições entre os consumers restantes — esse processo é chamado de **rebalance**. Durante um rebalance, o consumo pode pausar brevemente enquanto as atribuições são recalculadas. Estratégias mais modernas ( `CooperativeStickyAssignor` ) minimizam esse impacto, evitando revogar todas as partições de todos os consumers a cada rebalance (comportamento do assignor clássico, `RangeAssignor` ).

## 4.4. Offsets: o "ack" do Kafka

Diferente do ack por mensagem do RabbitMQ, no Kafka o consumer confirma seu progresso através do **commit de offset** — um número que diz "já processei tudo até aqui, nesta partição". Esse commit é armazenado em um tópico interno especial do próprio Kafka, chamado `__consumer_offsets`.

| Estratégia                                               | Comportamento                                                                  | Risco                                                                                                |
|----------------------------------------------------------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Auto-commit<br>( <code>enable.auto.commit=true</code> )  | O cliente confirma offsets automaticamente em intervalos (ex.: a cada 5s)      | Pode confirmar offset de mensagens ainda não totalmente processadas, causando perda em caso de falha |
| Commit manual síncrono<br>( <code>commitSync</code> )    | Você confirma explicitamente após processar, e o método bloqueia até confirmar | Mais seguro, porém mais lento                                                                        |
| Commit manual assíncrono<br>( <code>commitAsync</code> ) | Confirma sem bloquear, com callback de erro                                    | Maior throughput, requer tratamento cuidadoso de falhas de commit                                    |

"At least once" também existe aqui

Assim como no RabbitMQ, se o consumer processar uma mensagem mas cair *antes* de confirmar o commit do offset, na próxima vez que o consumer (ou outro do mesmo grupo) assumir aquela partição, ele vai reler a partir do último offset confirmado — reprocessando mensagens já tratadas. A mesma disciplina de **idempotência** do Projeto 7 de RabbitMQ se aplica aqui.

## 5. Garantias de Entrega e Semânticas de Processamento

### 5.1. As três semânticas possíveis

| Semântica            | O que significa                                                  | Como obter no Kafka                                                                                                                                  |
|----------------------|------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>At most once</b>  | Mensagem processada 0 ou 1 vez — pode perder, nunca duplica      | Commit do offset <i>antes</i> de processar (raramente desejável)                                                                                     |
| <b>At least once</b> | Mensagem processada 1 ou mais vezes — nunca perde, pode duplicar | Padrão mais comum: processar, depois confirmar o commit                                                                                              |
| <b>Exactly once</b>  | Mensagem processada exatamente 1 vez, mesmo com falhas           | Kafka Transactions API — mais complexo, usado tipicamente dentro do Kafka Streams ou em pipelines produtor+consumidor no mesmo processo transacional |

### 5.2. Exactly-once em mais detalhe

O Kafka oferece suporte nativo a semântica *exactly-once* através de **transações**, combinando producer idempotente com um protocolo de duas fases que agrupa a escrita de novos registros e o commit de offsets em uma única unidade atômica. Isso é especialmente poderoso em pipelines do tipo "ler de um tópico, transformar, escrever em outro tópico" (o padrão que o Kafka Streams usa internamente) — garantindo que, mesmo em caso de falha e reprocessamento, o resultado final seja como se cada mensagem tivesse sido processada exatamente uma vez.

# 6. Retenção e Compactação de Log

## 6.1. Políticas de retenção

Diferente de uma fila tradicional, o Kafka não apaga uma mensagem quando ela é lida. Em vez disso, cada tópico tem uma política de retenção configurável:

| Configuração                        | Efeito                                                                                               |
|-------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>retention.ms</code>           | Por quanto tempo manter os dados (padrão: 7 dias)                                                    |
| <code>retention.bytes</code>        | Tamanho máximo por partição antes de começar a descartar os dados mais antigos                       |
| <code>cleanup.policy=delete</code>  | Padrão: descarta dados mais antigos que o limite de retenção                                         |
| <code>cleanup.policy=compact</code> | Em vez de descartar por tempo, mantém apenas o <b>último valor de cada chave</b> — ver próxima seção |

## 6.2. Log Compaction (compactação)

Quando um tópico usa `cleanup.policy=compact`, o Kafka periodicamente remove registros antigos de uma mesma chave, mantendo apenas o valor mais recente — como uma tabela hash persistente, em vez de um histórico completo de eventos. É o padrão usado, por exemplo, pelo próprio tópico interno `__consumer_offsets`: não importa o histórico de commits antigos, só o offset mais recente de cada partição/grupo importa.

```
Antes da compactacao (chave: valor):
[K1:V1] [K2:V1] [K1:V2] [K3:V1] [K1:V3] [K2:V2]

Depois da compactacao:
[K3:V1] [K1:V3] [K2:V2]
(mantém so o ultimo valor de cada chave, ordem de insercao original preservada)
```

### Quando usar compactação

Útil para modelar "estado atual" via eventos — por exemplo, um tópico `usuarios.perfil` onde cada mensagem representa o perfil completo e atualizado de um usuário (chave = ID do usuário). Compactar esse tópico significa que ele nunca cresce indefinidamente: sempre reflete o estado mais recente de cada usuário, funcionando como uma espécie de banco de dados distribuído baseado em log.

## 7. Instalação e Ambiente de Desenvolvimento

### 7.1. Subindo o Kafka localmente com Docker (modo KRaft, sem ZooKeeper)

Usando a imagem oficial da Apache em modo combinado (broker + controller no mesmo processo, adequado para desenvolvimento local):

```
docker run -d --name kafka \
  -p 9092:9092 \
  -e KAFKA_NODE_ID=1 \
  -e KAFKA_PROCESS_ROLES=broker,controller \
  -e KAFKA_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093 \
  -e KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092 \
  -e KAFKA_CONTROLLER_LISTENER_NAMES=CONTROLLER \
  -e KAFKA_CONTROLLER_QUORUM_VOTERS=1@localhost:9093 \
  -e KAFKA_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT \
  -e KAFKA_INTER_BROKER_LISTENER_NAME=PLAINTEXT \
  -e CLUSTER_ID=MkU3OEVBNTcwNTJENDM2Qk \
  apache/kafka:latest
```

### 7.2. docker-compose.yml para projetos de estudo

```
version: "3.8"
services:
  kafka:
    image: apache/kafka:latest
    container_name: kafka-estudo
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@localhost:9093
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      CLUSTER_ID: MkU3OEVBNTcwNTJENDM2Qk
    volumes:
      - kafka_data:/var/lib/kafka/data

volumes:
  kafka_data:
```

### 7.3. Interface gráfica: Kafka UI

Diferente do RabbitMQ (que já vem com Management UI embutida via plugin), o Kafka não tem uma interface web oficial pronta. A opção mais usada pela comunidade para desenvolvimento local é o **Kafka UI** (projeto open source), adicionável facilmente via Docker Compose, apontando para o broker acima — ela mostra tópicos, partições, mensagens, consumer groups e seus offsets, cumprindo um papel parecido ao da Management UI do RabbitMQ.

### 7.4. Linha de comando: os scripts inclusos

O Kafka vem com scripts de linha de comando úteis para inspeção manual (dentro do container ou com o Kafka instalado localmente):

| Comando                                                                       | Para que serve  |
|-------------------------------------------------------------------------------|-----------------|
| <code>kafka-topics.sh --create --topic X --partitions 3 --replication-</code> | Criar um tópico |

|                                                                                                     |                                                                                     |
|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>factor 1 --bootstrap-server localhost:9092</code>                                             |                                                                                     |
| <code>kafka-topics.sh --list --bootstrap-server localhost:9092</code>                               | Listar tópicos existentes                                                           |
| <code>kafka-console-producer.sh --topic X --bootstrap-server localhost:9092</code>                  | Publicar mensagens manualmente via terminal                                         |
| <code>kafka-console-consumer.sh --topic X --from-beginning --bootstrap-server localhost:9092</code> | Consumir mensagens manualmente, lendo desde o início                                |
| <code>kafka-consumer-groups.sh --describe --group X --bootstrap-server localhost:9092</code>        | Ver o lag (atraso) de um consumer group — quantas mensagens ele ainda não processou |

## 8. Trabalhando com Kafka em Java

### 8.1. O cliente oficial: kafka-clients

Assim como o `amqp-client` para RabbitMQ, o Kafka tem uma biblioteca cliente oficial de baixo nível: `org.apache.kafka:kafka-clients`. Funciona identicamente em qualquer versão do JDK moderna, o que a torna igualmente adequada para comparar código entre diferentes versões do Java.

#### Maven

```
<dependency>
  <groupId>org.apache.kafka</groupId>
  <artifactId>kafka-clients</artifactId>
  <version>3.8.0</version>
</dependency>
```

### 8.2. Producer básico

```
import org.apache.kafka.clients.producer.*;
import java.util.Properties;

public class PedidoProducer {

    private static final String TOPIC = "pedidos.eventos";

    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");
        props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");
        props.put("acks", "all");
        props.put("enable.idempotence", "true");

        try (KafkaProducer<String, String> producer = new KafkaProducer<>(props)) {

            String chave = "pedido-42"; // garante ordem para o mesmo pedido
            String valor = "{\"pedidoId\":42,\"status\":\"CRIADO\"}";

            ProducerRecord<String, String> record =
                new ProducerRecord<>(TOPIC, chave, valor);

            producer.send(record, (metadata, exception) -> {
                if (exception == null) {
                    System.out.println("Enviado para particao " + metadata.partition()
                        + " no offset " + metadata.offset());
                } else {
                    exception.printStackTrace();
                }
            });
        }
    }
}
```

### 8.3. Consumer básico com commit manual

```
import org.apache.kafka.clients.consumer.*;
import java.time.Duration;
import java.util.Collections;
import java.util.Properties;
```

```

public class PedidoConsumer {

    private static final String TOPIC = "pedidos.eventos";

    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", "processamento-pedidos");
        props.put("key.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("value.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("enable.auto.commit", "false"); // commit manual
        props.put("auto.offset.reset", "earliest"); // le do inicio se nao houver offset salvo

        try (KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props)) {
            consumer.subscribe(Collections.singletonList(TOPIC));

            while (true) {
                ConsumerRecords<String, String> records = consumer.poll(Duration.ofMillis(500));

                for (ConsumerRecord<String, String> record : records) {
                    System.out.printf("particao=%d offset=%d chave=%s valor=%s\n",
                        record.partition(), record.offset(), record.key(), record.value());

                    // processar a mensagem aqui
                }

                if (!records.isEmpty()) {
                    consumer.commitSync(); // confirma o progresso apos processar o lote
                }
            }
        }
    }
}

```

## 8.4. Consumer com múltiplas instâncias (mesmo consumer group)

Basta rodar a mesma classe `PedidoConsumer` em processos separados, usando o mesmo `group.id`. O Kafka automaticamente distribui as partições entre as instâncias ativas — não é necessário nenhum código adicional, diferente do RabbitMQ onde múltiplos workers simplesmente competem pela mesma fila.

### Cuidado: mais consumers do que partições

Se você tem um tópico com 3 partições e sobe 5 instâncias do mesmo consumer group, 2 delas ficarão completamente ociosas — o Kafka não divide uma partição entre múltiplos consumers do mesmo grupo. O paralelismo máximo de um consumer group é limitado ao número de partições do tópico.

## 9. O Ecossistema Kafka (visão geral)

---

### 9.1. Kafka Streams

Biblioteca Java para processamento de streams diretamente sobre tópicos Kafka — permite transformações, agregações, joins e janelas de tempo (windowing) sem precisar de um cluster de processamento separado (como Spark ou Flink). É a ferramenta natural quando a lógica de negócio é "ler de um tópico, transformar, escrever em outro tópico", com garantias de exactly-once nativas.

### 9.2. Kafka Connect

Framework para integrar o Kafka com sistemas externos (bancos de dados, S3, Elasticsearch, etc.) sem escrever código customizado de producer/consumer — usando **conectores** prontos (source connectors puxam dados para o Kafka; sink connectors empurram dados do Kafka para fora). Muito usado para padrões como Change Data Capture (CDC), capturando mudanças de um banco relacional como um fluxo de eventos.

### 9.3. ksqlDB / Schema Registry

O **ksqlDB** permite escrever consultas estilo SQL diretamente sobre streams do Kafka. O **Schema Registry** (geralmente usado com Avro ou Protobuf) resolve um problema comum em produção: garantir que producers e consumers concordem sobre o formato/schema dos dados ao longo do tempo, com compatibilidade controlada entre versões.

#### Para o seu momento de aprendizado

Streams, Connect e ksqlDB são construídos *sobre* os conceitos fundamentais deste guia (tópicos, partições, offsets, producers, consumers). Vale dominar bem os fundamentos com o cliente puro ( `kafka-clients` ) antes de partir para essas ferramentas de mais alto nível — exatamente como você fez com `amqp-client` antes de considerar Spring AMQP.

# 10. Boas Práticas e Erros Comuns

| Prática                                                                               | Motivo                                                                                                                                                                          |
|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Escolher bem o número de partições desde o início                                     | Aumentar depois é possível, mas <b>muda o resultado do hash da chave</b> — mensagens da mesma chave podem passar a cair em partições diferentes, quebrando a ordenação esperada |
| Sempre definir uma chave quando a ordem importa                                       | Sem chave, não há garantia de que eventos relacionados fiquem na mesma partição                                                                                                 |
| Usar <code>acks=all</code> + <code>enable.idempotence=true</code> para dados críticos | Evita perda de dados e duplicação por retry de rede                                                                                                                             |
| Fazer commit do offset só depois de processar de verdade                              | Evita perder mensagens em caso de falha entre o auto-commit e o processamento real                                                                                              |
| Monitorar o consumer lag                                                              | Lag crescendo indica que o consumer está processando mais devagar do que a taxa de produção — sinal de que falta escalar                                                        |
| Tornar o processamento idempotente                                                    | At-least-once (o padrão mais comum) permite reproprocessamento de mensagens já tratadas                                                                                         |
| Não tratar o Kafka como um banco de dados de consulta direta                          | É um log sequencial — não existe "buscar pelo valor X" sem processar/indexar externamente (ou usar Streams/ksqlDB para isso)                                                    |

## Erro comum: assumir ordenação global

Uma dúvida frequente de quem vem do RabbitMQ: "a mensagem A foi publicada antes da B, então A sempre chega antes?" No Kafka, isso só é garantido se A e B tiverem a **mesma chave** (e portanto caírem na mesma partição). Entre partições diferentes, não há ordem garantida nenhuma — cada partição é lida de forma independente.

# 11. Apache Kafka vs. RabbitMQ vs. Amazon SQS/SNS

Retomando e aprofundando o comparativo já visto no guia de RabbitMQ, agora com mais detalhe técnico dos três lados.

| Critério                              | Kafka                                                                                                 | RabbitMQ                                                   | SQS / SNS                                                    |
|---------------------------------------|-------------------------------------------------------------------------------------------------------|------------------------------------------------------------|--------------------------------------------------------------|
| Modelo mental                         | Log distribuído, particionado, replicado                                                              | Broker tradicional com roteamento rico                     | Fila gerenciada (SQS) e pub/sub gerenciado (SNS)             |
| Após consumo                          | Mensagem permanece no log até expirar por retenção                                                    | Mensagem é removida da fila (ack)                          | Mensagem é removida (SQS) após ack                           |
| Replay de mensagens antigas           | Nativo — é a base do modelo (reprocessar offset)                                                      | Não nativo (Streams se aproxima)                           | Não suportado nativamente                                    |
| Múltiplos consumidores lendo tudo     | Nativo, via consumer groups independentes                                                             | Possível via fanout/topic (cada um com sua fila)           | Via SNS fan-out para múltiplas filas SQS                     |
| Roteamento sofisticado (por conteúdo) | Limitado — o producer decide o tópico/partição                                                        | Nativo e muito flexível (topic/headers exchange)           | Filtros de assinatura no SNS                                 |
| Ordem garantida                       | Por partição (via chave)                                                                              | Dentro de uma fila                                         | SQS Standard não garante; FIFO garante, com menor throughput |
| Throughput                            | Muito alto — projetado para milhões de eventos/s                                                      | Alto, mas geralmente inferior ao Kafka em cargas massivas  | Alto, gerenciado pela AWS                                    |
| Dead lettering                        | Não nativo — implementado manualmente                                                                 | Nativo (Dead Letter Exchange)                              | Nativo (DLQ do SQS)                                          |
| Operação                              | Historicamente complexa (hoje simplificada com KRaft), ou serviços gerenciados (Confluent Cloud, MSK) | Você opera o cluster (ou usa um gerenciado, ex. CloudAMQP) | Totalmente gerenciado pela AWS                               |
| Melhor para                           | Streaming de eventos, event sourcing, big data, replay, altíssimo throughput                          | Tarefas, comandos, orquestração, RPC, roteamento complexo  | Integração serverless dentro do ecossistema AWS              |

### A pergunta que resume a escolha

"Eu preciso reprocessar o histórico de eventos no futuro, ou tenho múltiplos times/sistemas que vão querer ler o mesmo fluxo de forma independente, em altíssimo volume?" — se sim, Kafka. "Eu preciso de roteamento sofisticado, prioridade, ou um modelo mais simples de tarefas que somem depois de processadas?" — RabbitMQ continua sendo a ferramenta certa. Os dois não são mutuamente exclusivos: é comum ver arquiteturas reais usando Kafka como espinha dorsal de eventos e RabbitMQ para filas de tarefas internas de um serviço específico.

## 12. Próximos Passos

---

Este guia cobre Apache Kafka em profundidade teórica — arquitetura, tópicos, partições, producers, consumers, consumer groups, offsets, garantias de entrega e o ecossistema ao redor. Seguindo o mesmo formato usado com RabbitMQ, os próximos materiais desta série serão:

1. **Guia de projetos práticos** — uma progressão de projetos pequenos em Java (8, 11, 17, 21, 25), cada um isolando um conceito: produtor/consumidor básico, múltiplas partições e consumer groups competindo, múltiplos grupos independentes lendo o mesmo tópico, chaves e ordenação, tratamento de falhas e reprocessamento, e um consumer "pronto para produção" com idempotência e graceful shutdown — o paralelo direto da progressão que você já fez com RabbitMQ.
2. **Guia visual** — diagramas de arquitetura e fluxogramas explicando o funcionamento interno de cada um desses projetos.